

FoodBridge: A Surplus Food Redistribution Matcher

Project Proposal for UCS503P

Thapar Institute of Engineering and Technology

Submitted to: **Mrs. Anushka**

Team Number: 3

Shikhar Goel(1024030916) (sgoel2_be24@thapar.edu)

Hardik Pajjala(1024030119) (hpajjala_be24@thapar.edu)

Ayush Shukla(1024030164) (ashukla_be24@thapar.edu)

Repository: (https://github.com/pajjala99/project_SE3)

1. Higher-order goal ...secondary framing

This project aims to reduce avoidable food waste in urban settings by shortening the co ordi-nation gap between organisations that generate surplus edible food and welfare organisations that can consume it within its usable window. While the topic touches on broader questions of food security and sustainability, the immediate engineering objective is narrower and concrete: to translate an informal, phone-and-messaging-based redistribution practice into a measurable, deployable software workflow in which allocation decisions are recorded, reviewable, and made against explicit criteria.

2. Time-to-value ...primary heuristic

We will deliver meaningful increments quickly by first building the end-to-end donation workflow — post, match, collect, confirm — with the simplest allocation rule that works, and validating it with users before improving the allocation itself.

We will prioritise fast feedback over perfection by using weekly iterations and CI-backed automation from the first week, so that the cost of releasing a change stays low throughout.

Success is defined by what can be delivered and measured in the first iteration — a donation that travels from posting to confirmed pickup with timestamps at every transition — and then improved on the evidence those timestamps provide.

3. Problem Statement

Restaurants, hostel and institutional mess halls, caterers, and event organisers routinely end up with surplus edible food, while nearby welfare organisations, shelters, and community kitchens struggle to source food on short notice. The food is safe and immediately consumable, but only for a short and shrinking window. Common pain points include:

Fragmentation: availability is announced through calls, WhatsApp groups, and personal contacts, so surplus arising outside a coordinator's existing network is never discovered at all.

Unoptimised allocation: where a listing does reach several organisations, the food goes to whoever replies first. Response speed correlates with volunteer availability, not with proximity, need, or the donation's remaining shelf life.

Low transparency: a donor who hands food over has no record of whether it was collected or wasted, and a recipient has no visibility of what is available nearby right now.

No measurement: because nothing is recorded, neither party can establish what was redistributed, and no baseline exists against which any improvement could be claimed.

Why this matters now (contextual relevance): the binding constraint is time, not willing-ness. A donation typically has a usable window of a few hours; a coordination process that takes most of that window converts a solvable logistics problem into waste. Even modest reductions in time-to-match therefore translate directly into food that is eaten rather than discarded.

4. Proposed Solution ... Software Proposition

4.1. Overview

Build a web-based "Surplus-to-Recipient" matching system with the following components:

Donor submission portal: a donor posts a surplus listing with food type, quantity in servings, expiry/pickup deadline, and pickup locality.

Recipient registration: welfare organisations register with their service area, daily serving capacity, and food-type constraints (for example, dietary restrictions or absence of refrigeration).

Automated matching: the system filters recipients that cannot accept the donation and ranks the remainder on proximity, urgency, and capacity fit, then offers the donation to the best-fit organisation rather than broadcasting it.

Status transparency: both parties track the donation through its lifecycle and are notified at each transition.

Admin/ops dashboard: performance summaries — servings redistributed, median time-to-match, donations lost to expiry.

4.2. Core workflow

1. Donor creates a listing and receives a donation ID immediately.
2. System validates completeness, geocodes the pickup locality, and filters recipients whose radius, capacity, or food-type constraints exclude them.
3. System scores the remaining candidates and offers the donation to the best-fit recipient.
4. Recipient accepts or declines within a response window; on decline or timeout the donation is re-offered to the next-best candidate.

5. Both parties advance the donation through Posted → Matched → Accepted → Picked Up → Completed, with a timestamp recorded at each transition.
6. A donation reaching its expiry deadline uncollected is marked Expired and counted against the system's own performance.

4.3. Operational constraints for an educational, tier-2/3 setting

Keep the solution usable on low-to-mid range devices via responsive web design; a donor's entire obligation must fit in one short form on a phone.

Avoid dependence on advanced research algorithms. Allocation uses a classical, library-backed assignment routine, not a novel or learned method.

Assume pickup transport is arranged by the two matched parties, as it is in existing informal arrangements. The system coordinates; it does not dispatch vehicles.

Design for deployability using common web hosting, a managed database, and free-tier authentication and geocoding services.

5. Solution Approach ... Engineering focus

This is an engineering course project, so we focus on scalable administrative and workflow aspects rather than novel algorithm research. The allocation logic is treated as one well-isolated component of a larger workflow system, not as the project's research contribution.

5.1. Matching and allocation ... non-research

Allocation is deliberately built from mature, well-understood parts:

Hard constraints are applied first, as filters. A recipient outside its declared service radius, unable to accept the food type, out of remaining capacity, or not yet verified is removed from consideration before any scoring occurs.

Ranking uses a transparent weighted score over four normalised components — proximity, urgency (time remaining before expiry), capacity fit, and a fairness term that damps repeat allocation to the same organisation. The formula is configuration, not hidden logic, and every offer stores its component values so that any allocation can be explained after the fact.

The first iteration assigns greedily: donations are processed in order of urgency and each is offered to its highest-scoring available recipient. This is simple, fast, and correct for the common case of sparsely arriving donations.

A later iteration upgrades to a global assignment across all simultaneously open donations, using the Hungarian method via `scipy.optimize.linear_sum_assignment`. We make no state-of-the-art claim: this is a 1955 algorithm available as a standard library primitive, chosen precisely because it carries no implementation risk.

Staff and recipients retain the decision. The system offers and ranks; it never silently discards a donation or forces an assignment. Declines are a normal path, not an error.

5.2. Web architecture ... web-based solution

A typical 3-tier web architecture, with the allocation logic isolated behind a service boundary so it can be replaced without touching the application:

Frontend: responsive React UI for donors and recipient organisations — submission form, offer inbox, status timeline, dashboard.

Backend API: Node.js with Express, handling authentication, submission, geocoding, status transitions, re-match on decline, and notification dispatch.

Matching service: a stateless Python component exposing a single operation — given open donations and eligible recipients, return a scored assignment. Testable in isolation with no database dependency.

Database: PostgreSQL storing users, donors, recipients, donations, match offers with their score components, pickups, and notifications. Spatial indexing supports radius-constrained candidate retrieval directly in the query rather than in application code.

5.3. CI/CD and fast delivery enabler

CI/CD will be used to:

Automatically build and run tests on every change, with the matching service's unit tests running on every commit since that is where defects are least visible by inspection.

Produce repeatable deployment artifacts to staging.

Enable safe and frequent releases of incremental features, so that the pilot can absorb changes mid-window without a manual release process.

6. Evaluation Criterion ... measurable and attributable

We define success using measurable metrics tied to the core workflow, all derived from database timestamps rather than from self-report.

6.1. Primary evaluation metric

Time-to-Match (TTM): time between donation submission and creation of an accepted match.

Target: median TTM $\leq$ 20 minutes during the pilot window.

Attribution: measured from database timestamps — the donation's created at_ against the responded at_ of the first accepted offer. Both are written by the system, not entered by users.

Why this metric: TTM is the quantity the system exists to reduce. It is fully within the software's control, unlike total pickup time, which depends on the transport arrangements of the two parties.

6.2. Secondary evaluation metrics

Rescue rate: percentage of posted donations reaching Completed before their expiry dead-line. This is the outcome measure; TTM is the mechanism.

Offer acceptance rate: percentage of first offers accepted rather than declined. A low rate indicates the scoring function is ranking poorly and needs re-weighting — so this metric evaluates the allocation logic specifically.

Mean donor-to-recipient distance: compared between the greedy and global assignment iterations on the same scenario set, to quantify what the upgrade actually buys.

User clarity: donor- and recipient-reported clarity of the status timeline using a simple 1–5 feedback question.

Reliability: availability of staging/production for login and submission during the pilot window (target $\geq 99\%$ uptime).

6.3. Pilot validation plan ... high-level

Recruit 8–12 pilot donors (campus mess halls, canteens, and local eateries) and 3–5 recipient organisations, simulating recipient roles internally if real organisations are unavailable within the iteration window.

Establish a baseline through short interviews on current practice — how surplus is announced today, and how long it typically takes to place — before the system is introduced.

Compare TTM and rescue rate against that baseline within the iteration window, and re-run the scenario set after each change to the scoring weights.

7. Scalability ... with foresight

1. **Scalable (theoretically):** the system should support growth in the number of donations, registered recipients, and concurrent dashboard usage by:

- decoupling components (frontend / backend / matching service),

- spatially indexing recipient locations so that candidate retrieval does not degrade linearly with the recipient count,

- enabling pagination and indexing for common queries,

- using stateless API and matching-service design.

2. **Operationally deployable (practically):** the system should run on common web host-ing:

- managed relational database with a spatial extension,

- containerized or platform-hosted deployment,

- CI/CD pipeline for staging/production,

- cached geocoding results, so that repeated addresses cost no external calls.

8. Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

Web framework for REST APIs and reactive pages (React, Express).

Managed relational database with a spatial extension for radius queries (PostgreSQL / PostGIS).

Token-based authentication as a managed service (Firebase Auth).

Geocoding service with a free tier, with results cached locally.

An assignment-problem solver available as a standard library routine (scipy.optimize.linear sum assign_

Test framework for unit and integration tests.

CI runner and staging deployment target.

We will use these mature components to focus on workflow design, correctness, and measurement rather than inventing new infrastructure. Notably, the one part of the system that could have been treated as a research problem — optimal allocation — is available off the shelf, which is what makes it affordable to include at all.

9. Project scope and deliverables ... iteration-friendly

9.1. Initial deliverable ... first iteration

Donor submission form + donation ID generation

Recipient registration with radius, capacity, and food-type constraints

Eligibility filtering and greedy best-fit offer to a single recipient

Status workflow: accept/decline, and the transitions through to Completed

Donor and recipient status pages showing the timeline/history

Basic logging and timestamps at every transition for TTM measurement

CI: build + backend and matching-service unit tests + linting

CD to staging with a single command or pipeline job

9.2. Subsequent deliverables

Global assignment across simultaneously open donations, with the greedy result retained for comparison

Automatic re-match on decline or response timeout, and expiry handling

Fallback path when no recipient is eligible: widen radius, then waitlist, then surface a manual list to the donor

Notification layer (email/SMS optional in a later iteration; starting with in-app notifications)

Admin dashboard for impact metrics, backlog, and expired-donation counts

Improved search/filtering and indexed queries for scale readiness

10. Risks and mitigations

Scoring weights appear arbitrary: define a fixed set of validation scenarios with expected outcomes *before* tuning, treat weights as recorded configuration, and use offer acceptance rate as an independent check on ranking quality.

Global assignment not completed within the schedule: the greedy matcher ships as a complete first iteration, and the upgrade is an isolated module behind a fixed service interface. The project remains demonstrable and measurable without it.

No eligible recipient for a donation: implement graduated fallback rather than failing — widen the radius, then waitlist and retry, then surface a manual list. Track how often this path is taken as a signal of recipient coverage.

Recipient legitimacy: automated verification is out of scope and we state this plainly. An administrator approval step precedes eligibility, and the limitation is documented rather than left implicit.

Data privacy / consent: minimise collected data; contact details are disclosed only after a confirmed match, and photographs of food are optional.

Geocoding rate limits during the pilot: cache all results in the database and pre-geocode the seed dataset ahead of any demonstration.

Evaluation measurement ambiguity: fix the status transition definitions and instrument their timestamps before development begins, so that TTM has a single unambiguous meaning throughout.

11. Summary

This project proposes a deployable web platform that closes the coordination gap between surplus food and the organisations that can use it, in a context where the binding constraint is time. It meets the course criteria by emphasising time-to-value — an end-to-end workflow with the simplest allocation rule ships first, and the allocation improves on evidence afterwards — by implementing CI/CD from the first iteration to support frequent safe releases, and by using measurable evaluation criteria drawn from system timestamps rather than self-report, with time-to-match as the primary metric. It remains focused on engineering workflow, correctness, and scalability readiness rather than research-driven novelty: the one optimisation component it contains is a classical algorithm consumed as a library primitive, and the project's contribution lies in formulating the domain problem correctly and measuring whether the resulting allocations are good.